

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 8

Roll no: 28 PRN No: 12411933 Name of Student: **Pragati S. Rakhunde**

Subject Name : Deep Learning

1. Aim

To implement a pre-trained BERT model for sentiment analysis using a sample dataset, fine-tune the model for binary text classification, evaluate its performance, and classify new text reviews as positive or negative.

2. Objectives

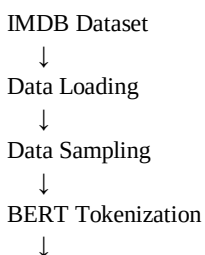
- To understand the concept of pre-trained Transformer models.
- To load and preprocess the IMDB movie review dataset.
- To tokenize text using a BERT tokenizer.
- To fine-tune a pre-trained BERT model for sentiment classification.
- To evaluate the model using accuracy, precision, recall and F1-score.
- To visualize model performance using training/validation loss and a confusion matrix.
- To test the trained model on custom text reviews.

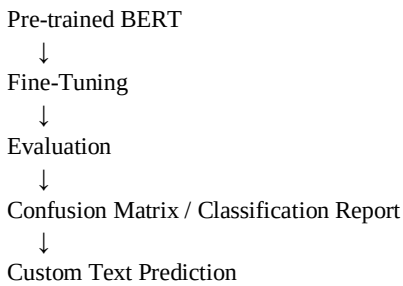
3. Introduction

BERT stands for Bidirectional Encoder Representations from Transformers. It is a Transformer-based language model that is pre-trained on large amounts of text. BERT learns contextual representations of words by considering information from both directions of a sentence.

In this practical, the pre-trained bert-base-uncased model is fine-tuned for binary sentiment classification. The model receives movie reviews from the IMDB dataset and predicts whether a review is positive or negative.

3.1 Overall Workflow





4. Software and Hardware Requirements

Software Requirements:

- Google Colab
- Python 3
- PyTorch
- Hugging Face Transformers
- Hugging Face Datasets
- Scikit-learn
- Matplotlib
- Seaborn

Hardware Requirement:

A GPU such as Google Colab T4 is recommended for faster BERT fine-tuning. The implementation can also run on CPU, but training will generally take longer.

5. Dataset Description

The IMDB Movie Review Dataset is used for sentiment analysis. Each review is associated with a binary sentiment label.

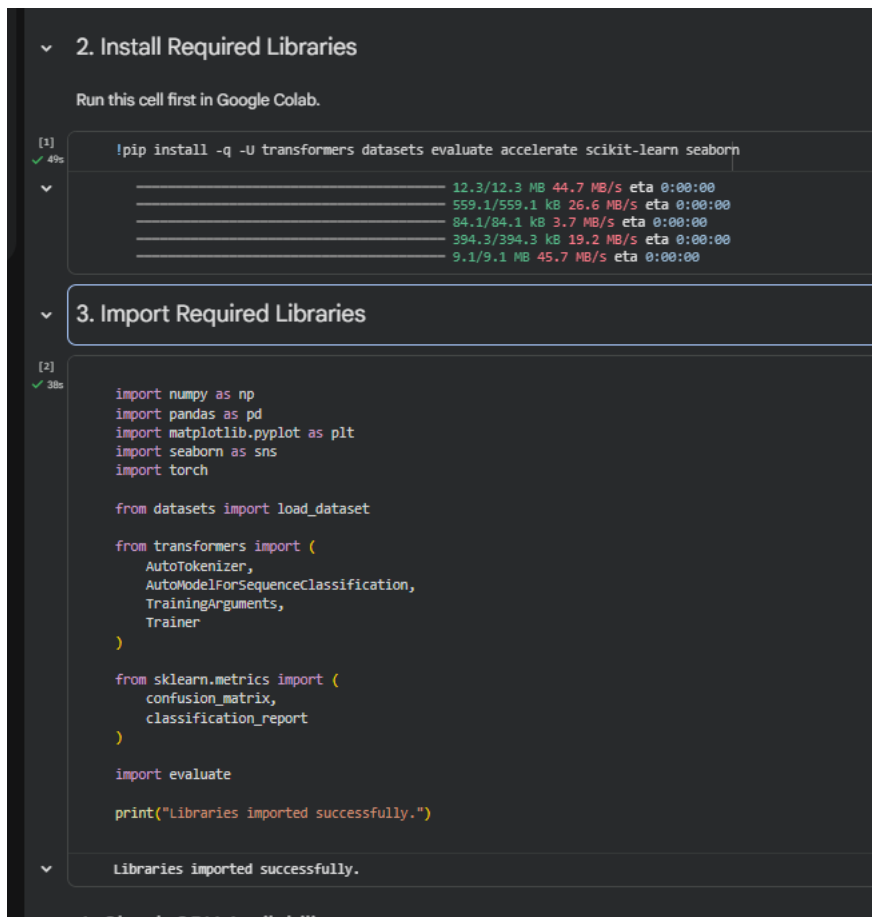
Label	Meaning
0	Negative
1	Positive

For this practical, a smaller subset is used to reduce execution time: 5,000 training reviews and 1,000 testing reviews.

6. Implementation

6.1 Install Required Libraries

```
!pip install -q -U transformers datasets evaluate accelerate scikit-learn seaborn
```



6.2 Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch
```

```
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
from sklearn.metrics import confusion_matrix, classification_report
import evaluate
```

6.3 Check GPU Availability

```
print("PyTorch version:", torch.__version__)
print("GPU available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("Running on CPU")
```

```
4. Check GPU Availability

BERT fine-tuning is faster when a GPU is available. In Google Colab, select Runtime → Change runtime type → T4 GPU.

[3]
✓ 0s
print("PyTorch version:", torch.__version__)
print("GPU available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("Running on CPU")

PyTorch version: 2.11.0+cu128
GPU available: True
GPU: Tesla T4
```

6.4 Load the IMDB Dataset

```
dataset = load_dataset("stanfordnlp/imdb")
print(dataset)
```

The dataset is downloaded automatically from Hugging Face. The repository identifier `stanfordnlp/imdb` is used for the current dataset-loading interface.

```
5. Load the IMDB Dataset

The IMDB dataset contains movie reviews with binary sentiment labels. The current Hugging Face repository identifier is stanfordnlp/imdb.

[4]
✓ 7s
dataset = load_dataset("stanfordnlp/imdb")
print(dataset)

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
README.md: 100% ██████████ 7.81k/7.81k [00:00<00:00, 640kB/s]

plain_text/train-00000-of-00001.parquet: reconstructing file: 100% ██████████ 21.0MB / 21.0MB, 2.04MB/s
plain_text/train-00000-of-00001.parquet: downloading bytes: ██████████ 20.9MB, 2.02MB/s
plain_text/test-00000-of-00001.parquet: reconstructing file: 100% ██████████ 20.5MB / 20.5MB, 2.00MB/s
plain_text/test-00000-of-00001.parquet: downloading bytes: ██████████ 20.4MB, 1.99MB/s
plain_text/unsupervised-00000-of-00001.p(...): reconstructing file: 100% ██████████ 42.0MB / 42.0MB, 4.09MB/s
plain_text/unsupervised-00000-of-00001.p(...): downloading bytes: ██████████ 41.9MB, 4.03MB/s

Generating train split: 100% ██████████ 25000/25000 [00:00<00:00, 64674.00 examples/s]
Generating test split: 100% ██████████ 25000/25000 [00:00<00:00, 60748.44 examples/s]
Generating unsupervised split: 100% ██████████ 50000/50000 [00:00<00:00, 99154.37 examples/s]

DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

6.5 Inspect a Sample Review

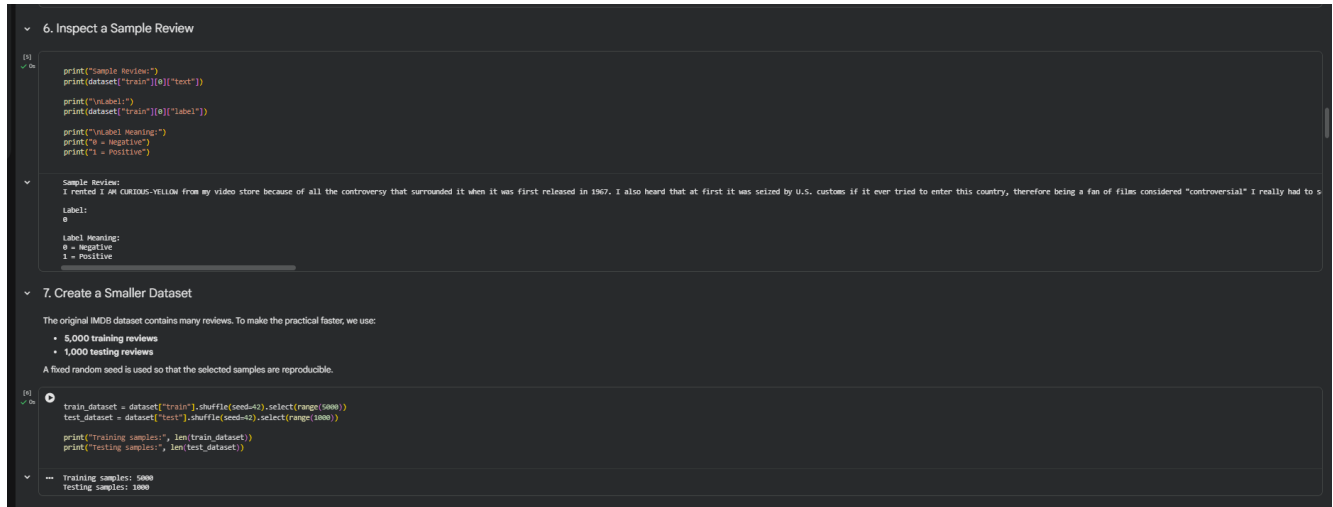
```
print("Sample Review:")
print(dataset["train"][0]["text"])

print("\nLabel:")
print(dataset["train"][0]["label"])
```

6.6 Create Training and Testing Subsets

```
train_dataset = dataset["train"].shuffle(seed=42).select(range(5000))
test_dataset = dataset["test"].shuffle(seed=42).select(range(1000))
```

```
print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))
```



The screenshot shows a Jupyter Notebook interface with two sections:

6. Inspect a Sample Review

```
In [ ]: print("Sample Review")
        print(dataset["train"][0]["text"])

        print("\nLabel:")
        print(dataset["train"][0]["label"])

        print("\nLabel Meaning:")
        print("0 = Negative")
        print("1 = Positive")
```

Sample Review:
I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was first released in 1967. I also heard that at first it was seized by U.S. customs if it ever tried to enter this country, therefore being a fan of films considered "controversial" I really had to s
Label:
0
Label Meaning:
0 = Negative
1 = Positive

7. Create a Smaller Dataset

The original IMDB dataset contains many reviews. To make the practical faster, we use:

- 5,000 training reviews
- 1,000 testing reviews

A fixed random seed is used so that the selected samples are reproducible.

```
In [ ]: train_dataset = dataset["train"].shuffle(seed=42).select(range(5000))
        test_dataset = dataset["test"].shuffle(seed=42).select(range(1000))

        print("Training samples:", len(train_dataset))
        print("Testing samples:", len(test_dataset))
```

Training samples: 5000
Testing samples: 1000

7. BERT Tokenization

A tokenizer converts raw text into numerical representations that can be processed by BERT. The maximum sequence length is set to 128 tokens. Longer reviews are truncated and shorter reviews are padded.

```
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

Tokenization demonstration:

```
text = "I loved this movie."
tokens = tokenizer.tokenize(text)

print("Original text:")
print(text)

print("\nTokens:")
print(tokens)

print("\nToken IDs:")
print(tokenizer.convert_tokens_to_ids(tokens))
```

9. Demonstrate BERT Tokenization

```
[8]
✓ Os
text = "I loved this movie."

tokens = tokenizer.tokenize(text)

print("Original text:")
print(text)

print("\nTokens:")
print(tokens)

print("\nToken IDs:")
print(tokenizer.convert_tokens_to_ids(tokens))
```

```
Original text:
I loved this movie.

Tokens:
['I', 'loved', 'this', 'movie', '.']

Token IDs:
[1045, 3866, 2023, 3185, 1012]
```

7.1 Tokenize the Dataset

```
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

train_tokenized = train_dataset.map(tokenize_function, batched=True)
test_tokenized = test_dataset.map(tokenize_function, batched=True)
```

10. Tokenize the Dataset

Each review is converted into:

- `input_ids` — numerical token IDs
- `attention_mask` — identifies actual tokens and padding
- `token_type_ids` — segment information used by BERT when applicable

Reviews are truncated or padded to a maximum length of 128 tokens.

```
[9]
✓ Os
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

train_tokenized = train_dataset.map(
    tokenize_function,
    batched=True
)

test_tokenized = test_dataset.map(
    tokenize_function,
    batched=True
)

print("Tokenization completed.")
```

```
Map: 100% ██████████ 5000/5000 [00:07<00:00, 806.50 examples/s]
Map: 100% ██████████ 1000/1000 [00:01<00:00, 853.00 examples/s]
Tokenization completed.
```

11. Inspect Tokenized Data

```
[10]
✓ Os
print(train_tokenized[0])

{'text': 'There is no relation at all between Fortier and Profiler but the fact that both are police series about violent crimes. Profiler looks crispy, Fortier looks classic. Profiler plots are quite simple. Fortier's plot are far more complicated... Fortier looks more like Prime Suspect, if we have to s...
```

8. Load the Pre-Trained BERT Model

The bert-base-uncased model is already pre-trained on general language data. For this task, a sequence classification head with two output classes is used.

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
```

```

num_labels=2
)

```

12. Load the Pre-Trained BERT Classification Model

`AutoModelForSequenceClassification` adds a classification head on top of BERT.

Since this is binary sentiment classification:

- 0 → Negative
- 1 → Positive
- `num_labels = 2`

```

[11] 66
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

print("BERT classification model loaded successfully.")

```

models.safetensors: reconstructing file: 100%
440MB / 440MB, 37.1MB/s

models.safetensors: downloading bytes:
415MB, 36.7MB/s

Loading weights: 100%
199/199 [00:00<00:00, 4879.24it/s]

[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

BERT classification model loaded successfully.

8.1 Move Model to GPU or CPU

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
print("Using device:", device)

```

9. Model Training and Fine-Tuning

Fine-tuning adapts the pre-trained BERT model to the specific sentiment classification task. The model parameters are updated using the labeled IMDB reviews.

9.1 Evaluation Metric

```

accuracy = evaluate.load("accuracy")

```

```

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return accuracy.compute(
        predictions=predictions,
        references=labels
    )

```

9.2 Training Arguments

```
training_args = TrainingArguments(
    output_dir="./bert_sentiment_results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_strategy="steps",
    logging_steps=100,
    load_best_model_at_end=True,
    report_to="none"
)
```

9.3 Create Trainer

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_tokenized,  
    eval_dataset=test_tokenized,  
    processing_class=tokenizer,  
    compute_metrics=compute_metrics  
)
```

9.4 Fine-Tune BERT

```
train_result = trainer.train()
```

```
16. Create the Trainer

trainer = trainer(
    model=model,
    argDict=training_args,
    train_dataset=train_tokenized,
    eval_dataset=eval_tokenized,
    processing_class=tokenizer,
    compute_metrics=compute_metrics
)

print("Trainer created successfully.")

Trainer created successfully.

17. Fine-Tune the BERT Model

This is the main training step. The pre-trained BERT model is fine-tuned using the labeled IMDB reviews.

train_result = trainer.train()

Epoch: 1250/1250 04:30, Epoch 2/2

epoch  Training loss  Validation loss  Accuracy
1      0.378780      0.375780      0.857000
2      0.254163      0.533751      0.851000

Writing model shards: 100%
Writing model shards: 100%

[Trainers] There were missing keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.weight', 'bert.embeddings.LayerNorm.bias', 'bert.encoder.layer.0.attention.output.LayerNorm.weight', 'bert.encoder.layer.0.attention.output.LayerNorm.bias', 'bert.encoder.layer.0.output.LayerNorm.weight', 'bert.encoder.layer.0.output.LayerNorm.bias']
[Trainers] There were unexpected keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.beta', 'bert.embeddings.LayerNorm.gamma', 'bert.encoder.layer.0.attention.output.LayerNorm.beta', 'bert.encoder.layer.0.attention.output.LayerNorm.gamma', 'bert.encoder.layer.0.output.LayerNorm.beta', 'bert.encoder.layer.0.output.LayerNorm.gamma']
```

10. Model Evaluation

```
results = trainer.evaluate()
```

```
print("Evaluation Results:")
for key, value in results.items():
    print(f"{key}: {value}")
```


18. Evaluate the Fine-Tuned Model

```

results = trainer.evaluate()

print("Evaluation Results:")
for key, value in results.items():
    print(f"{key}: {value}")

```

(125/125 00.0%)

Training Loss	Validation Loss	Epoch	Accuracy
0.234163	0.376670	2	0.857000

Evaluation Results:
eval_loss: 0.3766695559624811
eval_accuracy: 0.857

19. Display Training History

```

history = trainer.state.log_history
history_df = pd.DataFrame(history)

display(history_df)

```

	loss	grad_norm	learning_rate	epoch	step	eval_loss	eval_accuracy	eval_runtime	eval_samples_per_second	eval_steps_per_second	train_runtime	train_samples_per_second	train_steps_per_second	total_flos	train_loss
0	0.585708	25.559496	1.841690e-05	0.16	100	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	0.399240	36.670967	1.661690e-05	0.32	200	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	0.374515	5.216481	1.521690e-05	0.48	300	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	0.413757	11.722637	1.361690e-05	0.64	400	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	0.386491	4.292089	1.201690e-05	0.80	500	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	0.370780	17.003296	1.041690e-05	0.96	600	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	NaN	NaN	NaN	1.00	625	0.375786	0.857	9.0703	110.250	13.781	NaN	NaN	NaN	NaN	NaN

10.1 Training History

```

history = trainer.state.log_history
history_df = pd.DataFrame(history)
display(history_df)

```

10.2 Training Loss Graph

```

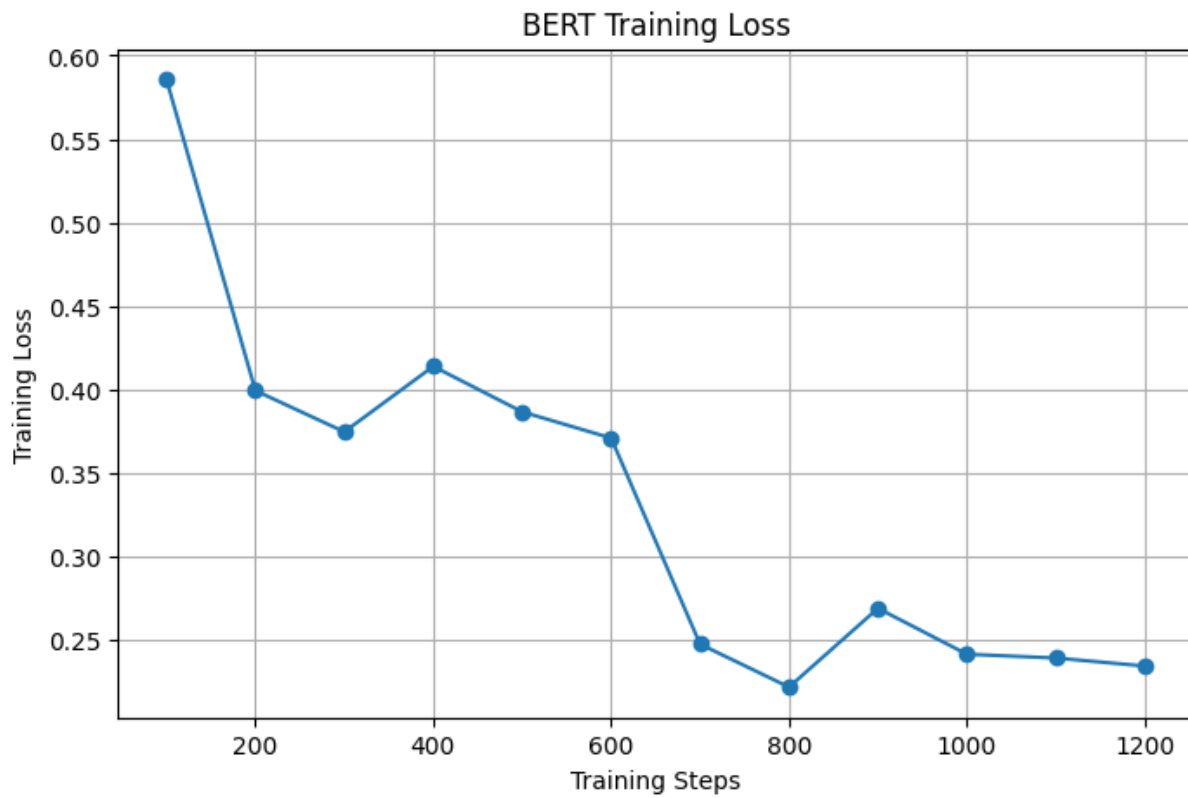
train_loss = history_df[history_df["loss"].notna()]

```

```

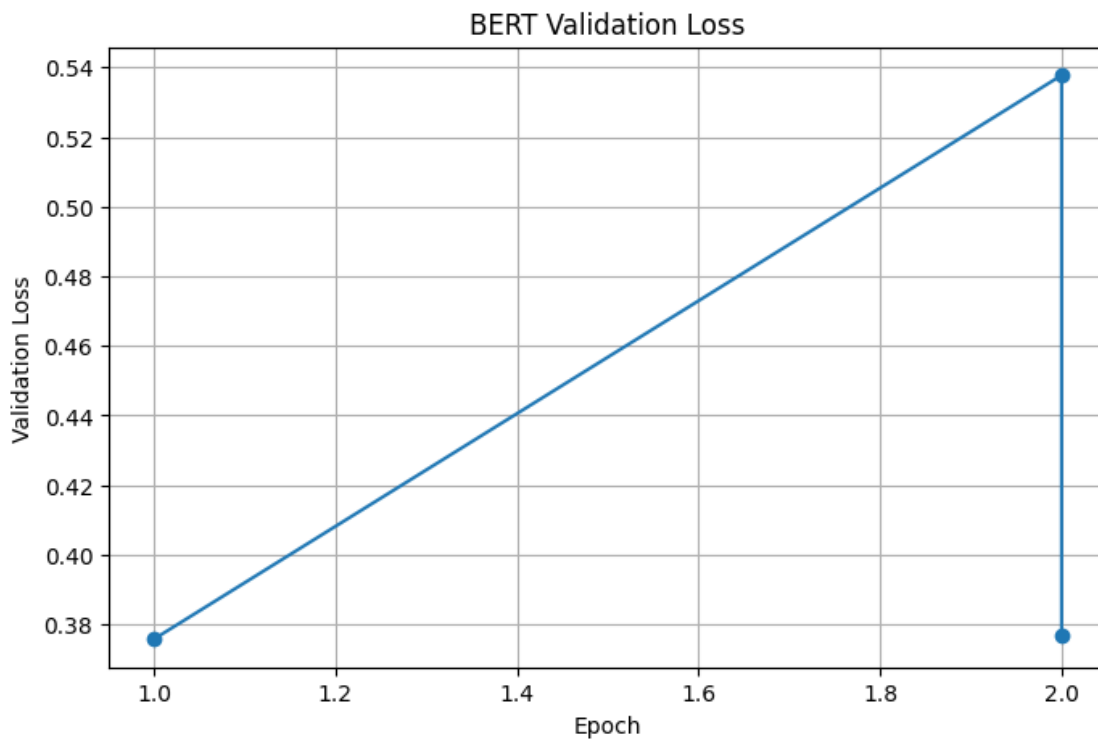
plt.figure(figsize=(8, 5))
plt.plot(train_loss["step"], train_loss["loss"], marker="o")
plt.xlabel("Training Steps")
plt.ylabel("Training Loss")
plt.title("BERT Training Loss")
plt.grid(True)
plt.show()

```



10.3 Validation Loss Graph

```
eval_loss = history_df[history_df["eval_loss"].notna()]\n\nplt.figure(figsize=(8, 5))\nplt.plot(eval_loss["epoch"], eval_loss["eval_loss"], marker="o")\nplt.xlabel("Epoch")\nplt.ylabel("Validation Loss")\nplt.title("BERT Validation Loss")\nplt.grid(True)\nplt.show()
```



11. Confusion Matrix

```
prediction_output = trainer.predict(test_tokenized)

predicted_labels = np.argmax(
    prediction_output.predictions,
    axis=1
)

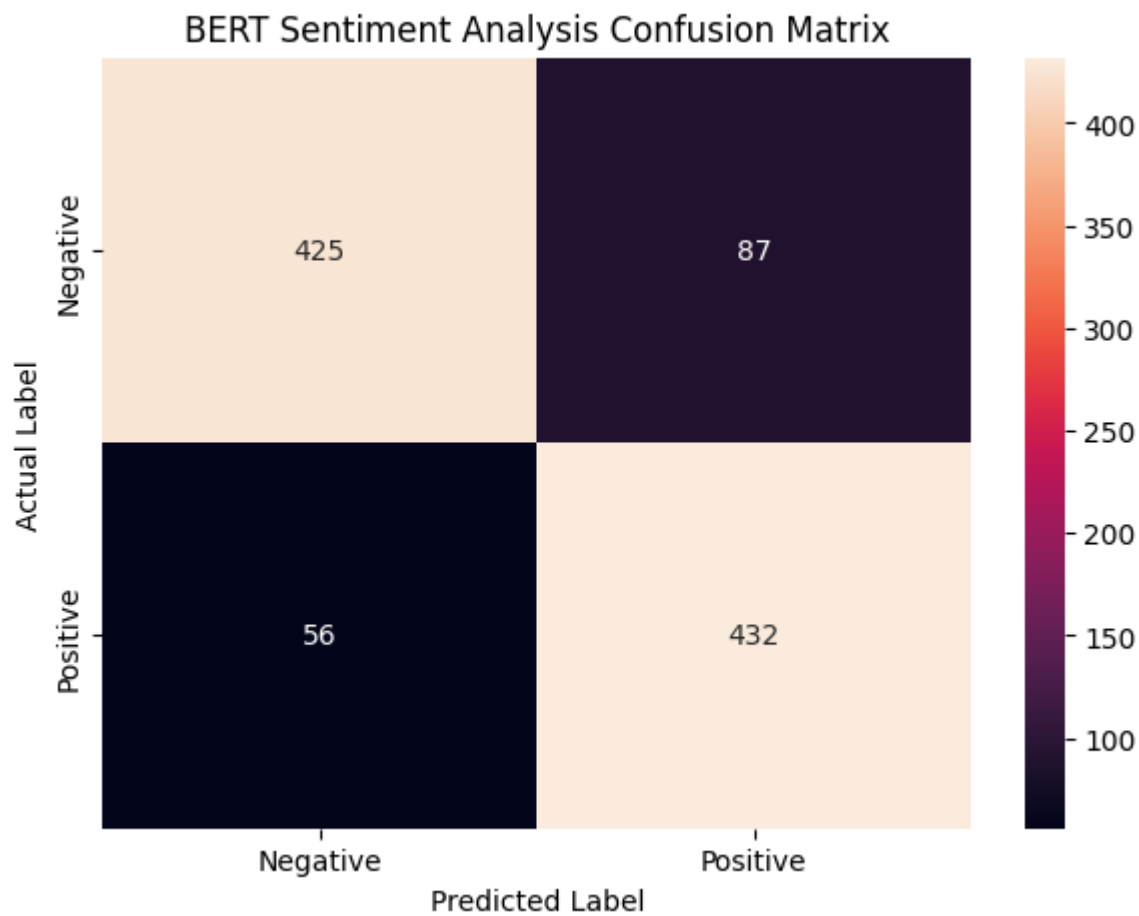
actual_labels = prediction_output.label_ids

cm = confusion_matrix(actual_labels, predicted_labels)
print(cm)

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("BERT Sentiment Analysis Confusion Matrix")
plt.show()
```



12. Classification Report

```
report = classification_report(  
    actual_labels,  
    predicted_labels,  
    target_names=["Negative", "Positive"]  
)  
  
print(report)
```

24. Classification Report

```
[24]  
✓ 0s  
report = classification_report(  
    actual_labels,  
    predicted_labels,  
    target_names=["Negative", "Positive"]  
)  
  
print(report)
```

	precision	recall	f1-score	support
Negative	0.88	0.83	0.86	512
Positive	0.83	0.89	0.86	488
accuracy			0.86	1000
macro avg	0.86	0.86	0.86	1000
weighted avg	0.86	0.86	0.86	1000

13. Prediction on Custom Reviews

The trained model can also be used to classify new reviews that were not present in the training dataset.

```
custom_reviews = [
    "This movie was absolutely fantastic. I really enjoyed it.",
    "The movie was boring and a complete waste of time.",
    "The acting was excellent and the story was very interesting.",
    "I hated this movie. It was terrible.",
    "It was an average movie, nothing special."
]

inputs = tokenizer(
    custom_reviews,
    padding=True,
    truncation=True,
    max_length=128,
    return_tensors="pt"
)

inputs = {key: value.to(model.device) for key, value in inputs.items()}

model.eval()

with torch.no_grad():
    outputs = model(**inputs)

predictions = torch.argmax(outputs.logits, dim=1)

for review, prediction in zip(custom_reviews, predictions):
    sentiment = "Positive" if prediction.item() == 1 else "Negative"
    print("Review:", review)
    print("Prediction:", sentiment)
    print("-" * 80)
```

```
predictions = torch.argmax(
    outputs.logits,
    dim=1
)

for review, prediction in zip(custom_reviews, predictions):
    sentiment = "Positive" if prediction.item() == 1 else "Negative"

    print("Review:", review)
    print("Prediction:", sentiment)
    print("-" * 80)
```

```
▼ ... Review: This movie was absolutely fantastic. I really enjoyed it.
      Prediction: Positive
      -----
      Review: The movie was boring and a complete waste of time.
      Prediction: Negative
      -----
      Review: The acting was excellent and the story was very interesting.
      Prediction: Positive
      -----
      Review: I hated this movie. It was terrible.
      Prediction: Negative
      -----
      Review: It was an average movie, nothing special.
      Prediction: Negative
      -----
```

14. Save the Fine-Tuned Model

```
save_path = "./bert_sentiment_model"
```

```
trainer.save_model(save_path)
```

```
tokenizer.save_pretrained(save_path)
```

```
print("Model saved at:", save_path)
```

▼

26. Save the Fine-Tuned Model

The fine-tuned BERT model and tokenizer can be saved for future use.

[26]
✓ 6s

▶

```
save_path = "./bert_sentiment_model"

trainer.save_model(save_path)
tokenizer.save_pretrained(save_path)

print("Model saved at:", save_path)
```

▼

Writing model shards: 100% 1/1 [00:06<00:00, 6.56s/sh]

Model saved at: ./bert_sentiment_model

27. Conclusion

A pre-trained BERT model was successfully implemented for sentiment analysis using the IMDB movie review dataset. The text data was tokenized using the BERT tokenizer, and the pre-trained `bert-base-uncased` model was fine-tuned for binary classification.

The model was evaluated using accuracy, loss, a confusion matrix, precision, recall, and F1-score. Finally, custom movie reviews were supplied to demonstrate sentiment prediction on unseen text.

Key Learning Outcomes

15. Results and Discussion

The pre-trained BERT model was successfully fine-tuned for binary sentiment analysis using the IMDB movie review dataset. The model performance is evaluated using accuracy, precision, recall, F1-score and a confusion matrix.

***	precision	recall	f1-score	support
Negative	0.88	0.83	0.86	512
Positive	0.83	0.89	0.86	488
accuracy			0.86	1000
macro avg	0.86	0.86	0.86	1000
weighted avg	0.86	0.86	0.86	1000

16. Conclusion

A pre-trained BERT model was implemented successfully for sentiment analysis. The IMDB dataset was tokenized using a BERT tokenizer, and the `bert-base-uncased` model was fine-tuned for binary classification. The trained model was evaluated using multiple classification metrics and a confusion matrix. Custom movie reviews were also classified to demonstrate the practical use of the fine-tuned model.